



CRC Course on Modelling

Professor Dr. Colin Atkinson, U Mannheim
Dr.-Ing. Erik Burger, KIT
Professor Dr. Ralf Reussner, KIT

Overview on Course Content



- Object-oriented Modelling:
 - Principles: Classes, Objects, inheritance, composition, association
 - Unified Modelling Language (UML) and Object Constraint Language (OCL)
- Model-Driven Software Development
 - Meta-Modelling, Meta Object Facility (MOF)
 - Deep Modelling
 - Transformations
- Orthographic Software Modelling and the Single Underlying Model (SUM)
- Vitruvius and the Virtual Single Underlying Model (V-SUM)

Overview on Course

Further Meta-Data



- Blocked Course Sept 9th -13th at KIT in presence
- Lectured by Colin Atkinson, Erik Burger, Ralf Reussner
 - Object oriented Modelling with the Unified Modelling Language
 - Object Constraint Language
 - Meta-Modelling with The Meta Object Facility
 - DeepModelling
 - Orthographic Software Modelling with Single Underlying Models
 - Vitruvius and Virtual Single Underlying Models

Model - Example

“Original”
Locomotive



Model
Lego Locomotive



Model Definition

- A model is an: (i) Abstraction of the modelled entity, (ii) With a given abstraction aim, (iii) Leaving out details not relevant to the abstraction, (iv) To simplify analysis/comprehensibility/...

Representation Feature („Abbildungsmerkmal“)

Models are always models of something, i.e., depictions, representations of natural or artificial originals, which can themselves be models.

Reduction Feature („Verkürzungsmerkmal“)

Model in general do not capture all attributes of the original they present, but only those that seem relevant to the respective model creators and/or model users.

Pragmatic Feature („Pragmatisches Merkmal“)

Models are not assigned to their original unequivocally per se. They fulfil their substitution function (a) for particular subjects, recognizing or acting, using models, (b) in particular intervals of time and (c) under restriction to particular mental or actual operations.

[Stachowiak 1973, pp. 131]

Buzz Group

Question:

➤ Why is this a model?



- Abstraction?
- Correspondence?
- Purpose?

Why Using Models?

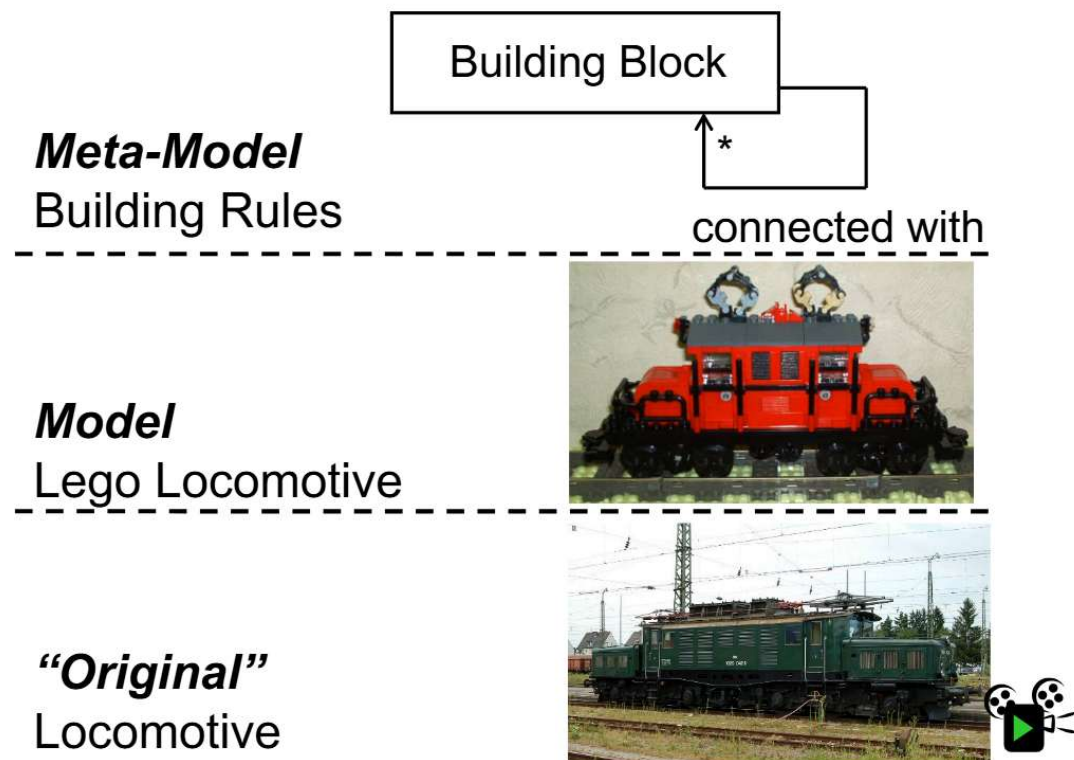
- Analysability: Reduced complexity allows (automatic) analyses
- Cope with Complexity: Models increase understandability through increased abstraction level
- Usable by domain experts (via Domain Specific Languages (DSLs))
- Reuse of domain abstractions
- Increased communication efficiency
- Always consistent documentation, if model is documentation

Overview

- Software Architecture
- **Key Concepts of View-based Modeling**
 - Model
 - **Metamodel**
 - View Points , View Types, Views
- Palladio Component Model (PCM) as a Modeling Language used by Palladio Approach
- Architectural View Points of Palladio
 - Structural View Points
 - Component Repository
 - Assembly
 - Behavioral View Points
 - Inner-component
 - Inter-component
 - Scenario Behavior
 - Deployment View Point
 - Decision View Point
- Conclusion / Summary

Model & Meta-Model

- How to specify a model?
- We *model* the model
- Result is a *meta-model*



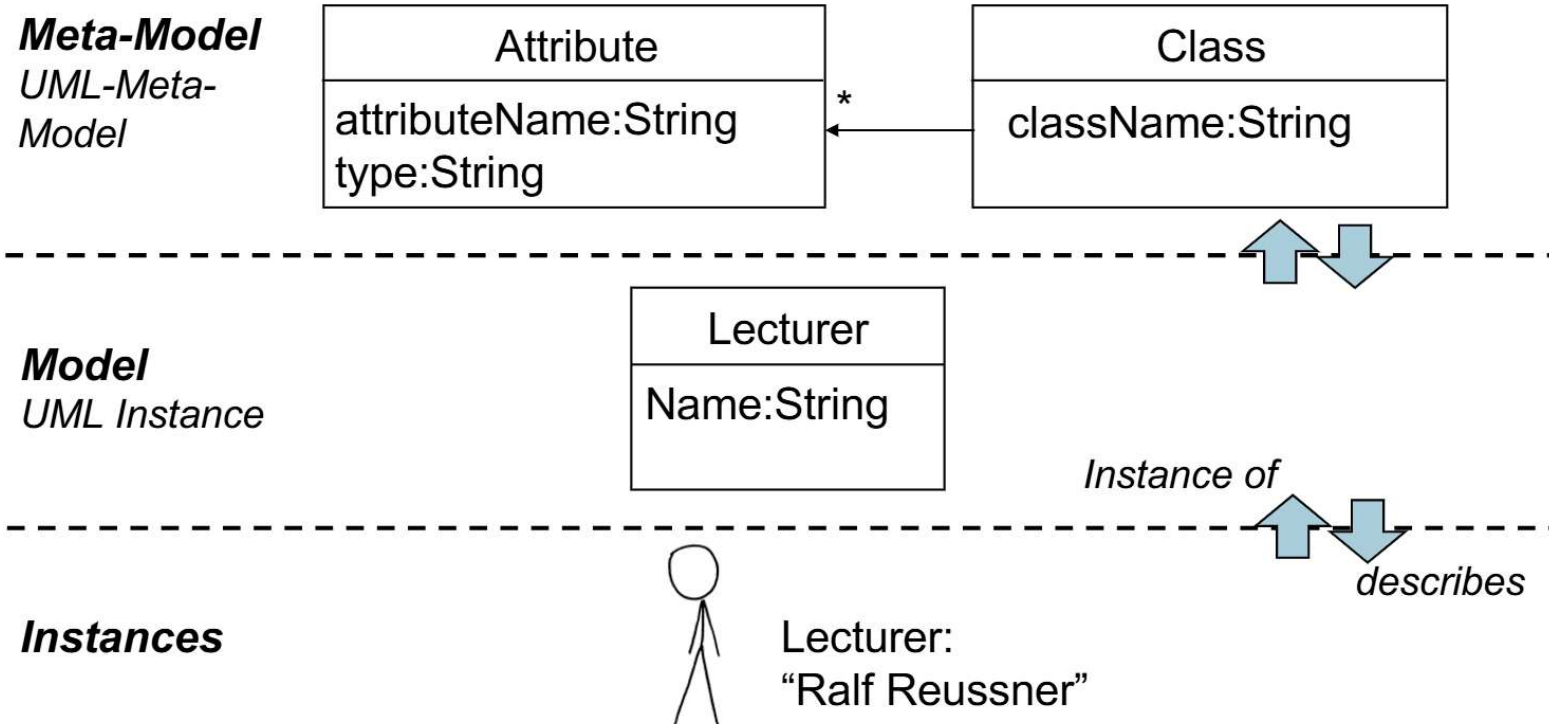
Is this really me?



- Hopefully not!
- This is just a bunch of arranged figures / pixels on the projector
- Again, this is a ***model*** of your lecturer
- Always keep in mind what you are talking about
- Don't confuse modelling levels



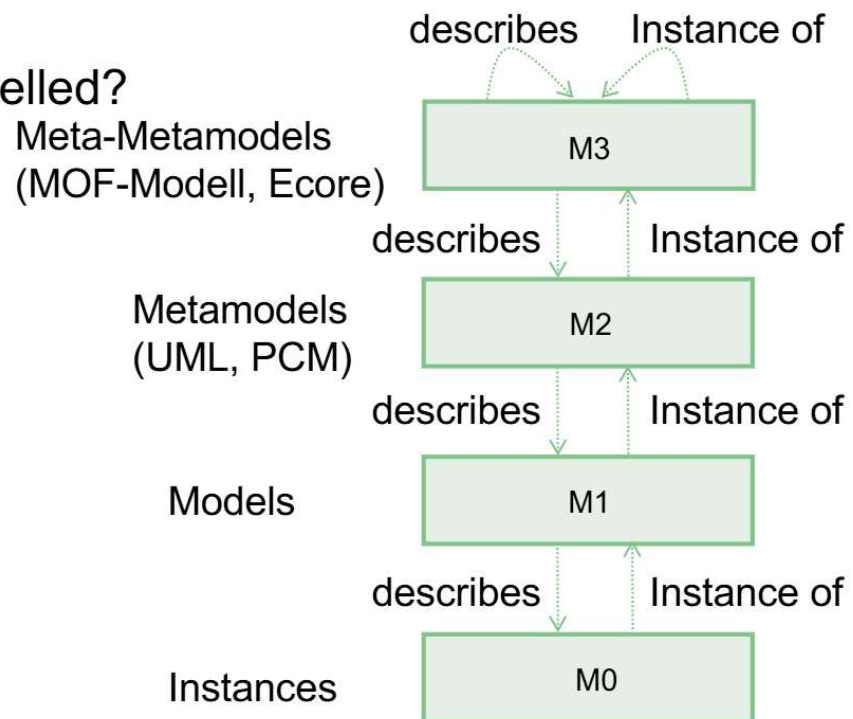
Meta-Model & Model & Instances - Example



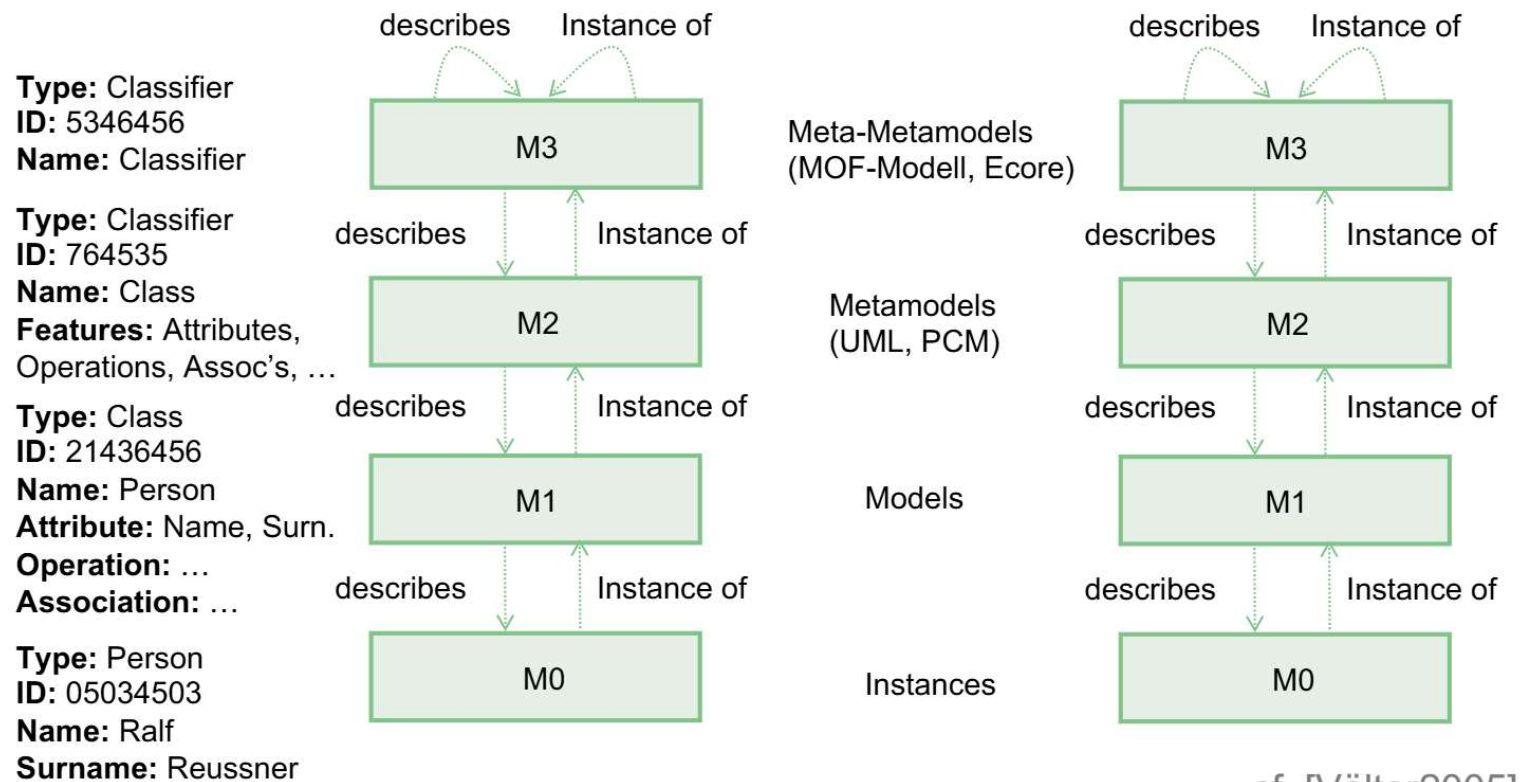
A **metamodel** is a precise definition of the parts and rules needed to create valid models.

Meta-Model-Levels

- A meta-model is *also a model*
- How is the meta-model then being modelled?
- Using a Meta-Meta-Model



Meta-Model-Levels – Example



cf. [Völter2005]

Meta-Model – Specification

A metamodel covers an abstract syntax, at least one concrete syntax, and static and dynamic semantics.

Abstract Syntax

Elements and their relations
independent of representation

Concrete Syntax

Representation of model-
Instances, e.g., on storage
devices

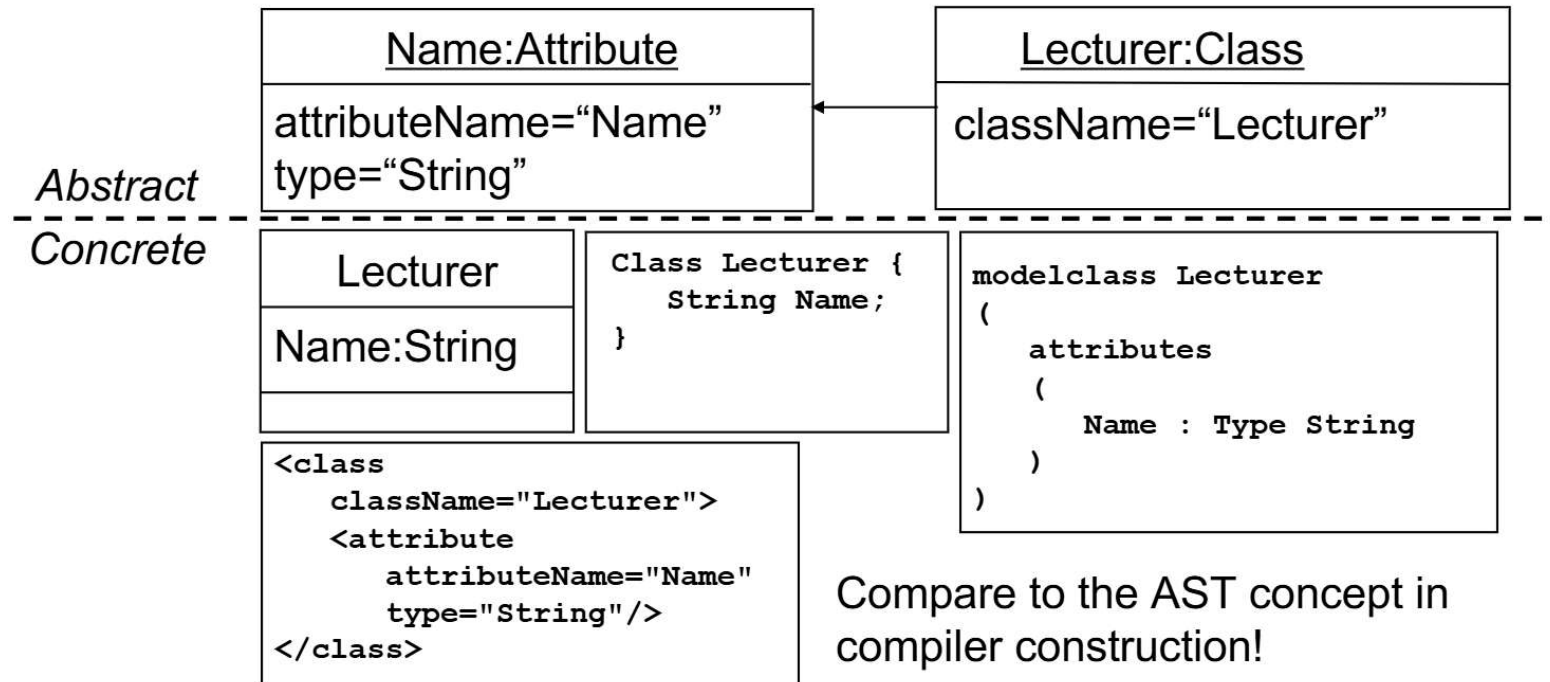
Static Semantics

Semantics evaluable without
executing the model

Dynamic Semantics

Description of behaviour of the
model

Abstract vs. Concrete Syntax



- There can be many concrete syntaxes for an abstract syntax
 - Different types, textual, graphical, tabular ...

Domain-specific Language (DSL)

- **Domain-specific languages (DSLs)** express domain concepts, with the goal of reducing the complexity in the modeling of systems.
- A DSL is defined by an **abstract syntax** and **a mapping to at least one concrete syntax**.
 - The abstract syntax describes the structure of a language in a way that is independent of its actual depiction or encoding. The abstract syntax of DSLs is usually not defined by a grammar but by a *meta-model*.
 - A concrete syntax defines the textual or graphical depiction or encoding of elements of the abstract syntax.
- Media Store example
 - An abstract syntax: A song has the properties title, artist, and year.
 - Concrete syntaxes for this abstract syntax:
 - A list that displays the information in textual form
 - A graphical representation as dots on a timeline
 - ID3 container format as a standard for description of audio files

Model-driven Approach

Conceptually ...

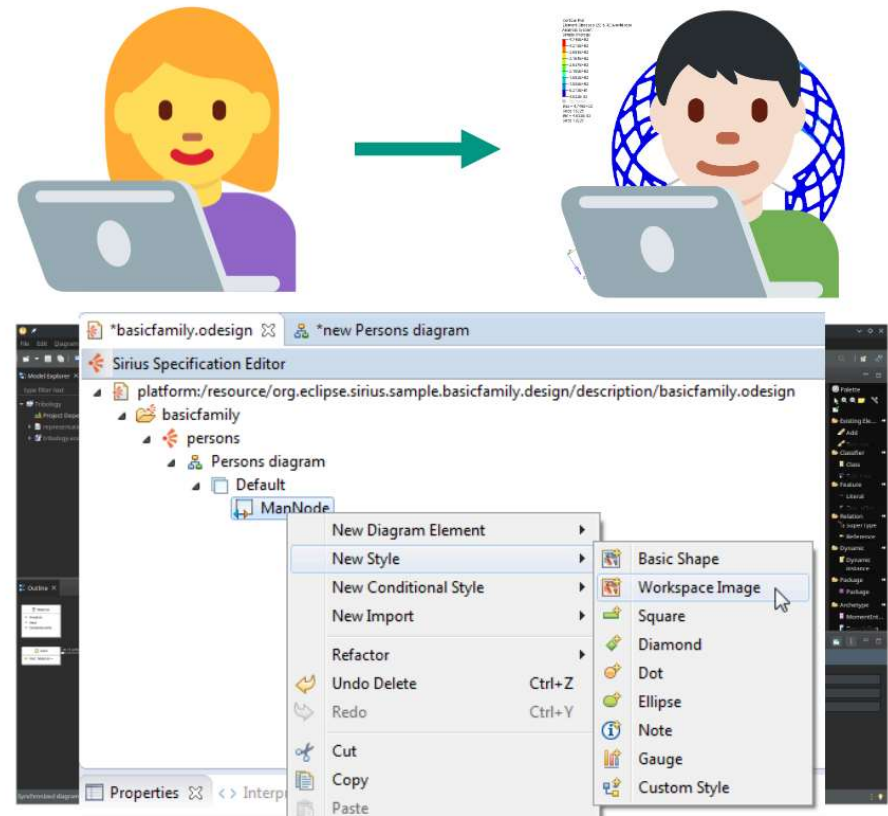
- a model is a representation of a natural or artificial *original* for a given purpose
- in Convide, (*cyber-physical*) systems are the originals, and development is the purpose
- a CPS is represented by multiple models (e.g., SysML diagrams, program code, CAD models, ...)
- the set of conventions for creating models is called **metamodel**

Technically ...

- the Meta Object Facility (MOF) is an ISO standard for model-driven development
- the Eclipse Modeling Framework (EMF) is MOF-compatible
- with EMF, developers can create
 - metamodels
 - model transformations
 - software for engineers

Example

1. Engineer Alice wants to model tribological properties of a car brake
2. Alice asks developer Bob to create a tool for her needs
3. Developer Bob creates a metamodel in the Eclipse Modelling Framework and calls it *Tribology Description Language (TDL)*
4. Bob creates a TDL editor
5. Alice uses this editor to model the car brake



Metamodels

Purpose

- a metamodel is a representation of models for a given purpose (e.g., tribology metamodel)
- metamodel $\approx$ language, thus has syntax and semantics
- metamodels, like languages, can be general-purpose or domain-specific
- metamodels are part of standards (e.g., SysML)

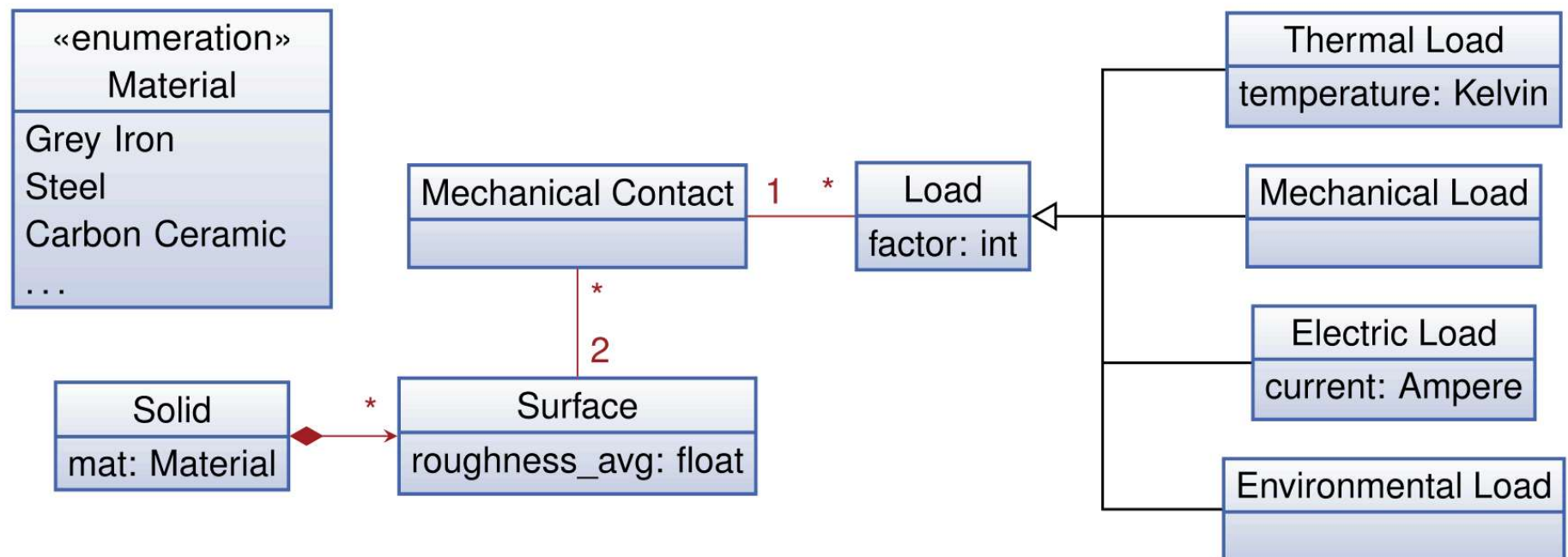
Author

- developers can create domain-specific languages (DSLs) and metamodels
- metamodels can be extracted from existing standards/file formats/. . . (e.g., SimuLink)

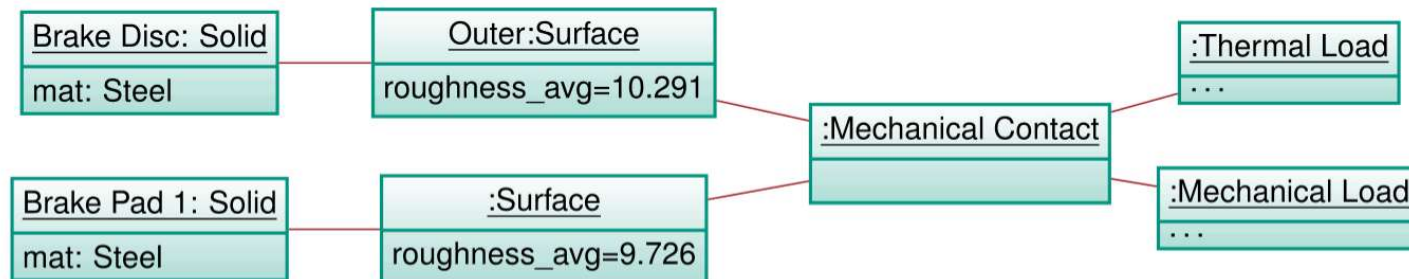
Object-oriented paradigm

- the abstract syntax of metamodels is usually defined as a class structure

Example: Tribology Metamodel



Example: Tribological Model of a Disc Brake



```

<solid name="Brake Disc">
  <mat>Steel</mat>
  <surface name="Outer">
    <roughness_avg>10.291</roughness_avg>
  </surface>
</solid>
    
```

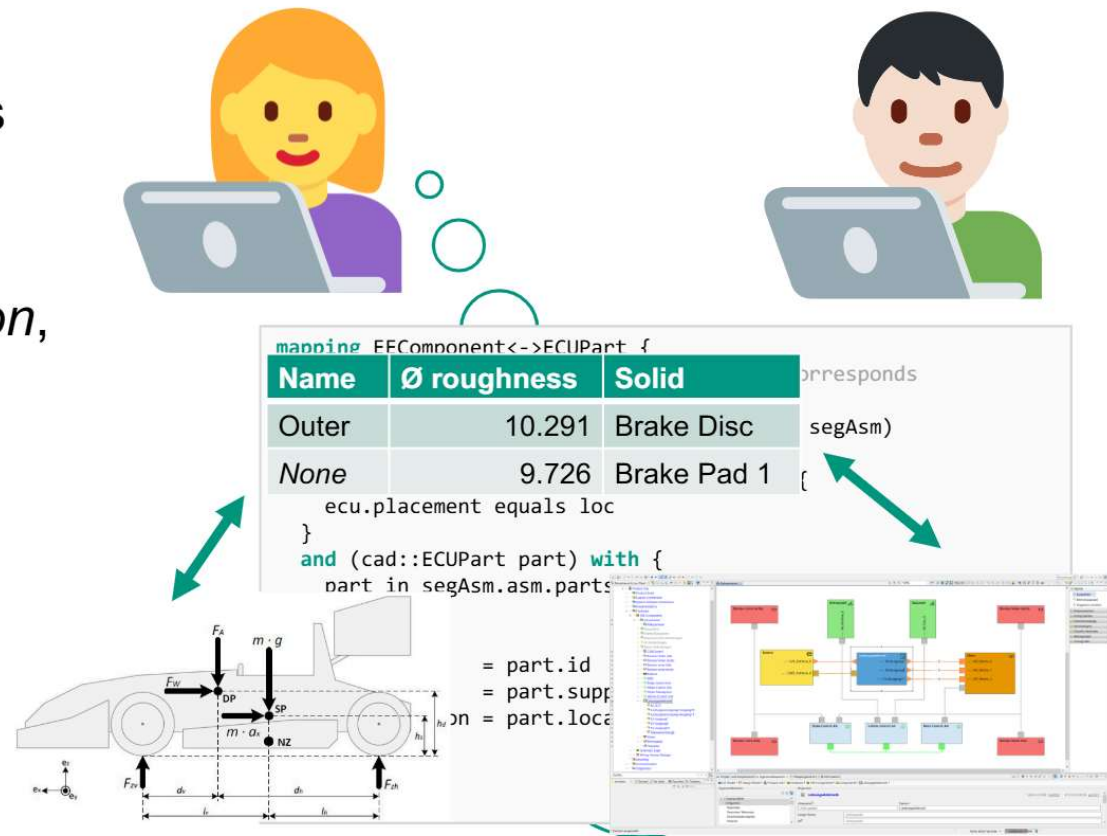
Surfaces

Name	Ø roughness	Solid
Outer	10.291	Brake Disc
None	9.726	Brake Pad 1

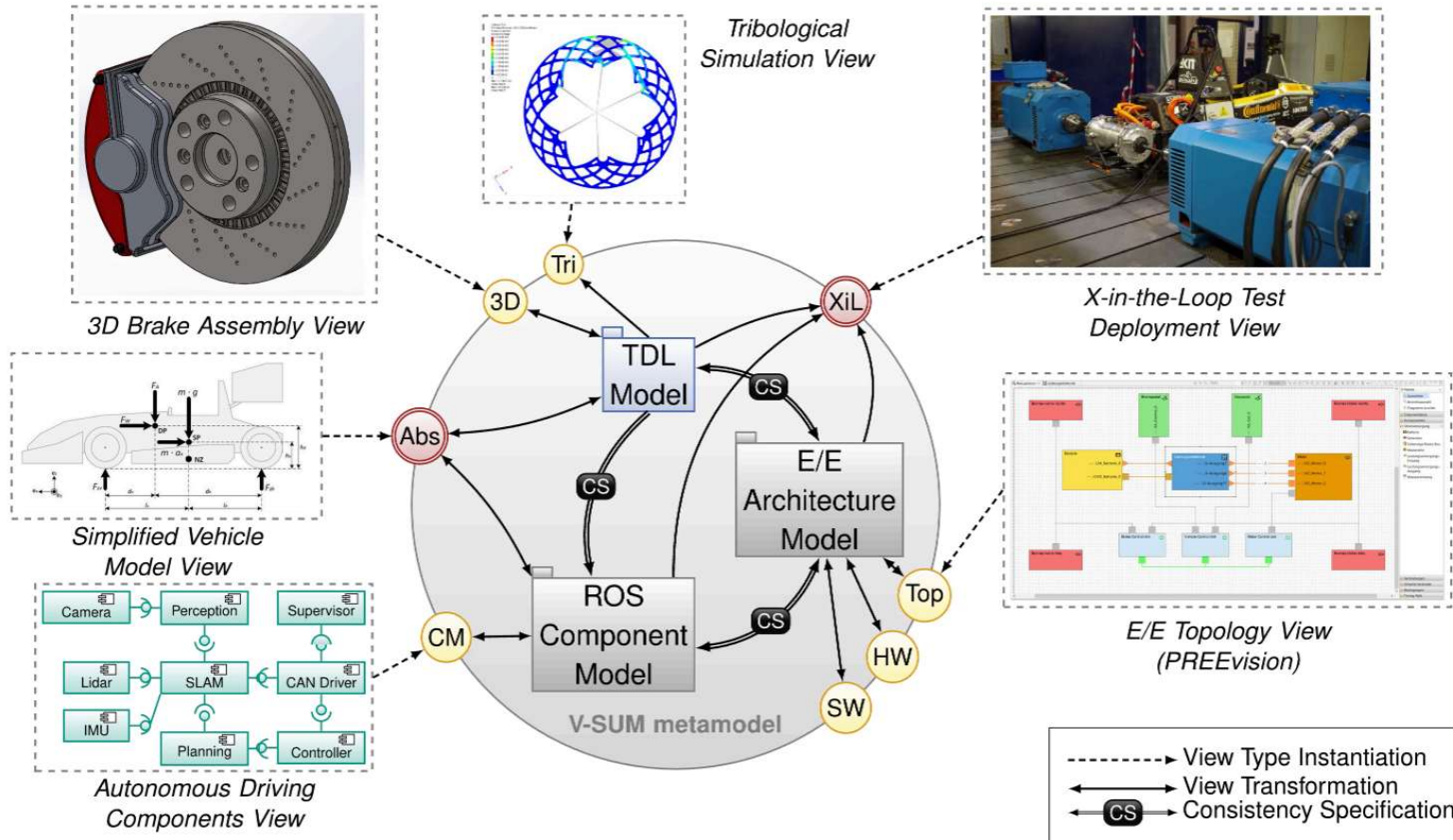
Different concrete syntaxes for the same model

Example: Consistency between Heterogeneous Models

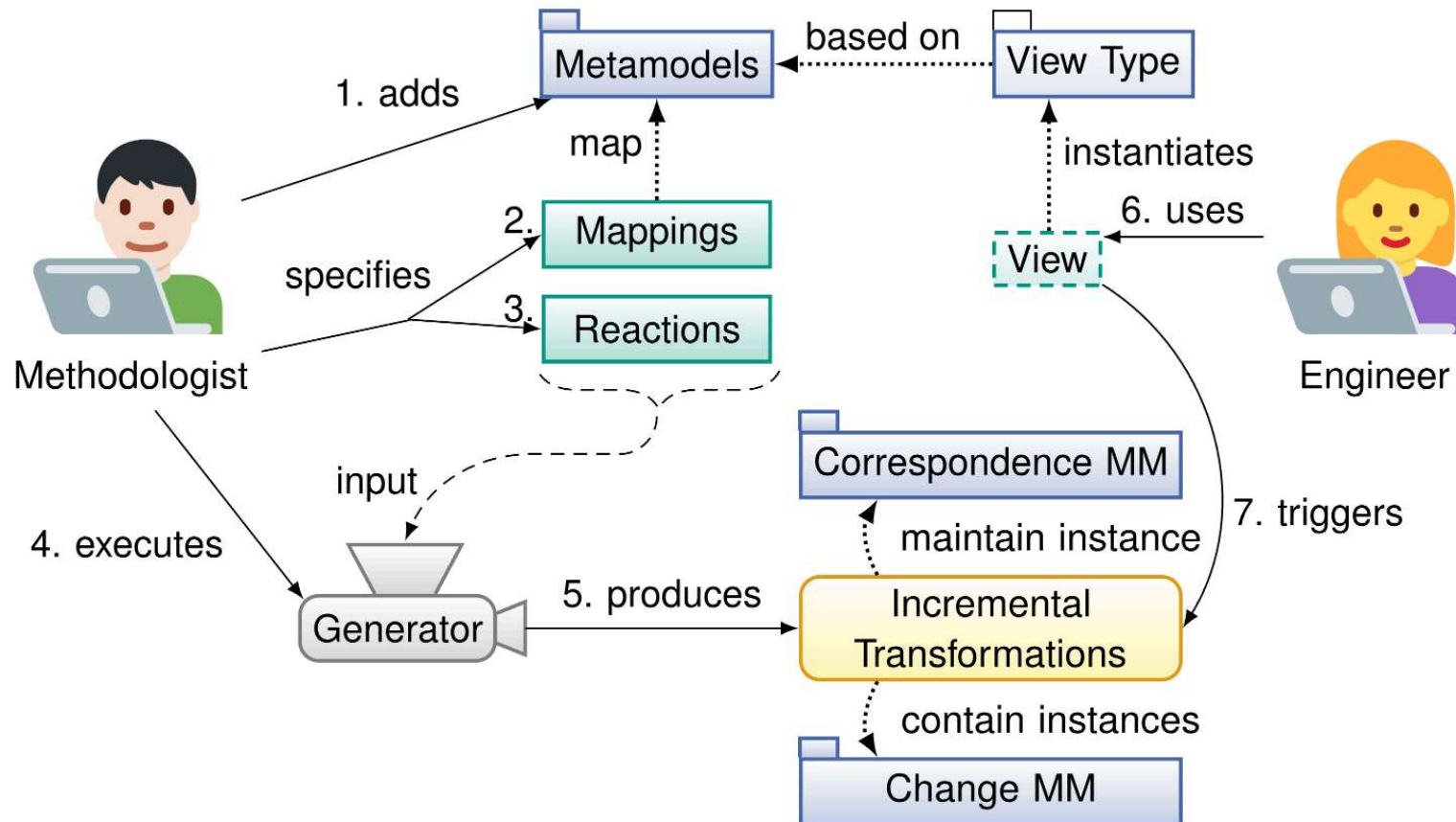
1. Alice wants to combine her TDL model with other, existing models
2. Alice decides to use Vitruvius
3. Alice asks developer Bob
4. Bob creates a Vitruvius *application*, writes the consistency specifications, and integrates the tooling
5. Alice uses Vitruvius to keep her models consistent



Vitruvius



Consistency Preservation with Vitruvius



Core principles of Vitruvius

Conceptual

- Virtual single underlying model
- Projective views
- incremental consistency specification
- change-based execution of consistency preservation

Technical

- research prototype based on the EclipseMoelling Framework
- heavy usage of model transformations
- not (yet) a user-friendly tool with GUI
- interaction needs programming skills